GRUPO 7

# SISTEMA BANCARIO

Modelado orientado en objetos con python

Curso: Lenguaje de Programacion II

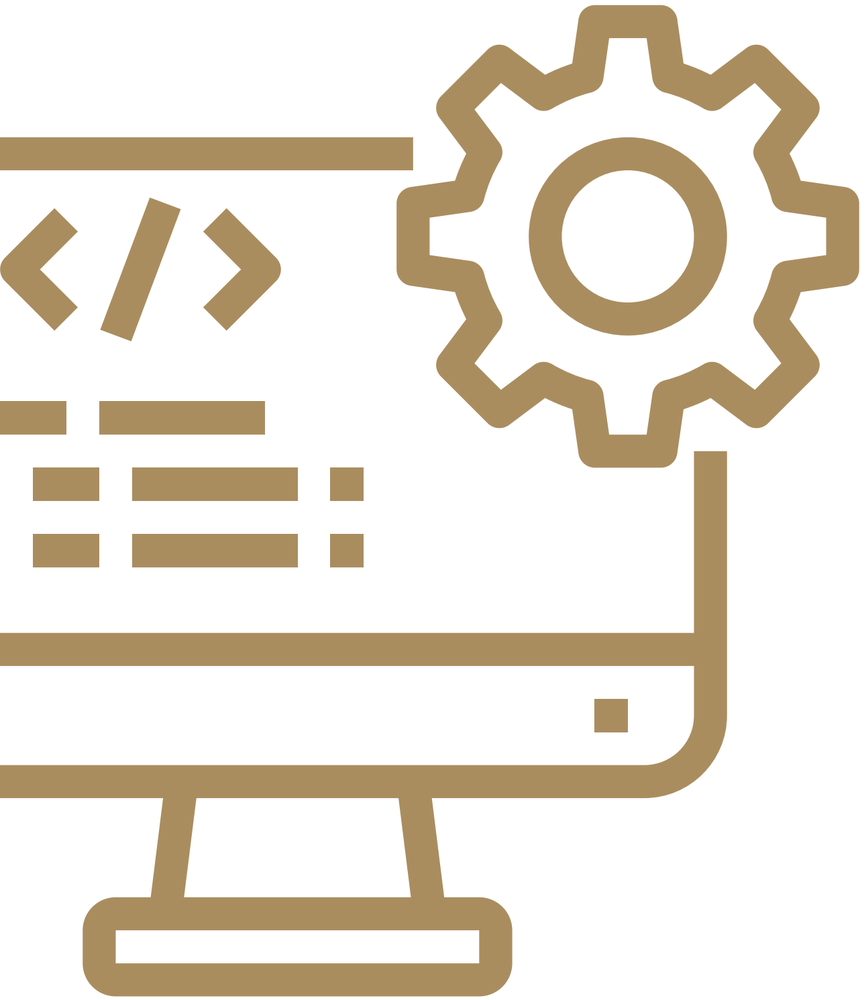


# INTEGRANTES

GRUPO 7

| | |
|---|---|
| OXSA QUISPE LAURA ISABEL | @lys-dotcom |
| ALAGON VICENTE, NAOMI ARACELY | @alagon-naomi |
| QUINTO PINEDO, ORLANDO ALFREDO RUBEN | @Rubs-5 |
| DE LA TORRE CONTRERAS, ADOLFO YOEL | @yoelADOLFO0712 |





# CONTENIDO


01. Introducción

02. Proposito del Simulador

03. Diagrama de clases

04. Conceptos POO aplicados

05. Clases

06. Modulo de interfaz de Usuario

# 1. INTRODUCCIÓN

El presente trabajo detalla el diseño, la lógica y la implementación del sistema computacional "Banco Nacional del Ahorro".

**1**

## PROBLEMA

La gestión financiera requiere consistencia estricta: evitar saldos negativos no autorizados y fraudes en las transacciones.

**2**

## PROPUESTA

Desarrollar un núcleo bancario en Python aplicando Programación Orientada a Objetos avanzada (encapsulamiento privado, herencia y polimorfismo).

**3**

## Ingeniería Colaborativa

Desarrollo modular en equipo utilizando Git y GitHub mediante ramas individuales y pruebas reproducibles en Jupyter Notebooks.

```
============================================================
        SISTEMA BANCARIO - BANCO NACIONAL DEL AHORRO
============================================================

------------------ MENU ------------------
 1. Cargar datos de ejemplo
 2. Registrar cliente
 3. Abrir cuenta
 4. Depositar
 5. Retirar
 6. Transferir
 7. Ver estado de cuenta (movimientos)
 8. Ver clientes y sus cuentas
 9. Cierre de mes (intereses y comisiones)
10. Reporte general del banco
 0. Salir
```

# 2. PROPOSITO DEL SIMULADOR

## Objetivo principal:

El objetivo principal de este desarrollo es construir un programa robusto, seguro y escalable, aplicando estrictamente los conceptos fundamentales de la Programación Orientada a Objetos (POO).



## Explicación del sistema simulado:

El sistema actúa como un administrador central que gestiona la interacción entre las personas y sus productos financieros. Para ello, el programa permite registrar usuarios (Clientes) y vincularlos con diferentes productos bancarios (Cuentas). Se han diseñado dos tipos de productos con reglas de negocio distintas:

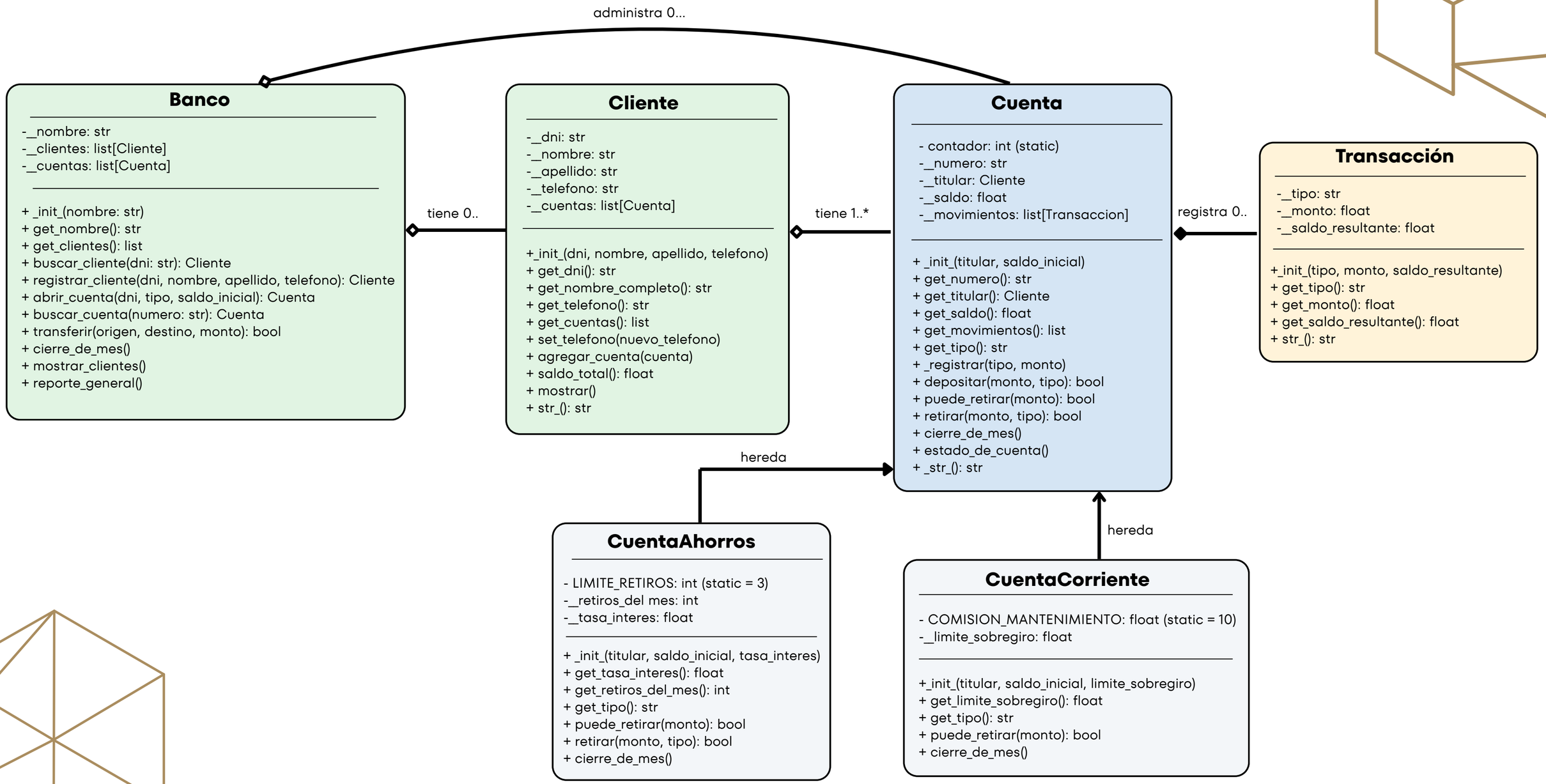
## Cuentas de Ahorros:

Orientadas a la rentabilidad, generando intereses a favor del cliente al finalizar el mes.

## Cuentas Corrientes:

Orientadas a la liquidez, permitiendo al cliente realizar retiros mayores a su saldo, a cambio del cobro de una comisión de mantenimiento mensual.

# 3. DIAGRAMA DE CLASES



# 4. CONCEPTOS POO APLICADOS

### Encapsulamiento

Los atributos críticos son privados (ej. __saldo, __movimientos). Su modificación está estrictamente controlada mediante Getters y reglas de validación.

### Herencia

La clase padre Cuenta centraliza el código común, heredando sus atributos y métodos a las subclases CuentaAhorros y CuentaCorriente.

### Polimorfismo

El método cierre_de_mes( ) es sobrescrito (Override). El sistema lo ejecuta de forma general, pero cada cuenta reacciona distinto (ganando intereses o cobrando comisiones).

### Abstracción

Operaciones como transferir( ) ocultan la complejidad interna (retiro, validación, depósito y registro histórico) detrás de una sola instrucción.



# 5. CLASES

Clase Cliente

Clase Cuenta

Clase Banco

Clase CuentaCorriente

Clase Transacción

Clase CuentaAhorro

# CLASE CLIENTE

El código de la clase cliente representa computacionalmente al usuario del banco y gestiona sus productos financieros.

- **Protección de Datos (Encapsulamiento)**: Todos los atributos son estrictamente privados (__dni, __nombre, __apellido, __telefono, __cuentas) para evitar alteraciones externas malintencionadas.
- **Asociación:** Aplica la relación "Cliente tiene Cuentas", inicializando una lista maestra en su constructor para almacenar todos los productos financieros asociados a esa persona.

```python
class Cliente:
    def __init__(self, dni, nombre, apellido, telefono=""):
        self.__dni = dni
        self.__nombre = nombre
        self.__apellido = apellido
        self.__telefono = telefono
        self.__cuentas = []

#  GETTERS
    def get_dni(self):
        return self.__dni

    def get_nombre_completo(self):
        return self.__nombre + " " + self.__apellido

    def get_telefono(self):
        return self.__telefono

    def get_cuentas(self):
        return self.__cuentas
```

# CLASE CLIENTE

**Validación de Entradas:** La modificación de datos es segura. El método set_telefono( ) incluye una regla estricta que solo acepta datos numéricos de exactamente 9 dígitos.

**Gestión Financiera:**

- agregar_cuenta( ): Permite vincular nuevas cuentas (Ahorros o Corriente) al usuario.
- saldo_total( ): Bucle que recorre todos los productos del cliente para calcular y sumar su patrimonio total.

**Reportes:** Los métodos mostrar( ) y __str__ () generan vistas detalladas y resumidas del estado general del cliente para la consola.

```python
#  SETTERS
    def set_telefono(self, nuevo_telefono):
        # Validacion: un celular en Peru tiene 9 digitos
        if nuevo_telefono.isdigit() and len(nuevo_telefono) == 9:
            self.__telefono = nuevo_telefono
        else:
            print("Telefono no valido: debe tener 9 digitos.")

#  METODOS
    def agregar_cuenta(self, cuenta):
        self.__cuentas.append(cuenta)

    def saldo_total(self):
        total = 0
        for cuenta in self.__cuentas:
            total += cuenta.get_saldo()
        return total

    def mostrar(self):
        print(f"\nCliente: {self.get_nombre_completo()}  |  DNI: {self.__dni}"
              f"  |  Tel: {self.__telefono}")
        if len(self.__cuentas) == 0:
            print("    (no tiene cuentas)")
        for cuenta in self.__cuentas:
            print("  ", cuenta)
        print(f"    Saldo total: S/ {self.saldo_total():.2f}")

    def __str__(self):
        return f"{self.__dni} - {self.get_nombre_completo()} ({len(self.__cuentas)} cuenta/s)"
```

# CLASE TRANSACCIÓN

El código de la clase Transaccion sirve para registrar y dar formato a los movimientos bancarios (como depósitos o retiros).

- __init__: Guarda de forma segura (privada) el tipo de transacción, el monto y el saldo resultante.
- __str__: Organiza y alinea visualmente esos datos junto con la moneda para mostrarlos ordenadamente en pantalla.
- Getters (get_*): Permiten consultar los datos protegidos de la transacción de forma controlada.

```python
class Transaccion:

    def __init__(self, tipo, monto, saldo_resultante):
        self.__tipo = tipo
        self.__monto = monto
        self.__saldo_resultante = saldo_resultante

    def __str__(self):
        return (f"{self.__tipo:<24} S/ {self.__monto:>10.2f}"
                f" Saldo: S/ {self.__saldo_resultante:>10.2f}")

    def get_tipo(self):
        return self.__tipo

    def get_monto(self):
        return self.__monto

    def get_saldo_resultante(self):
        return self.__saldo_resultante
```

# CLASE BANCO

La clase Banco actúa como el administrador central de todo el sistema bancario, gestionando las operaciones entre las personas y sus cuentas.

- **Base de Datos Interna:** Utiliza el encapsulamiento para proteger sus listas maestras (__clientes y __cuentas), manteniendo el control absoluto de todos los registros financieros.
- **Gestión Segura (Validaciones):** Sus métodos de registro y apertura verifican reglas clave antes de instanciar objetos. Por ejemplo, registrar_cliente( ) comprueba que el DNI tenga exactamente 8 dígitos y evita crear usuarios duplicados.

```python
class Banco:
    def __init__(self, nombre):
        self.__nombre = nombre
        self.__clientes = []     # LISTA de objetos Cliente
        self.__cuentas = []      # LISTA de objetos Cuenta

    def get_nombre(self):
        return self.__nombre

    def get_clientes(self):
        return self.__clientes
```

```python
def registrar_cliente(self, dni, nombre, apellido, telefono=""):
    # Validaciones
    if not (dni.isdigit() and len(dni) == 8):
        print("DNI no valido: debe tener 8 digitos.")
        return None
    if self.buscar_cliente(dni) is not None:
        print("Ya existe un cliente con ese DNI.")
        return None
    cliente = Cliente(dni, nombre, apellido, telefono)
    self.__clientes.append(cliente)
    print(f"Cliente {cliente.get_nombre_completo()} registrado correctamente.")
    return cliente
```

# CLASE BANCO

- **Transacciones Seguras:** El método transferir( ) implementa una regla: primero intenta ejecutar el retiro en la cuenta de origen y, solo si fue exitoso (hay saldo suficiente), procede a depositar en el destino.

- **Polimorfismo:** Al ejecutar cierre_de_mes(), el banco simplemente recorre su lista maestra de cuentas. Cada objeto "sabe" cómo comportarse (sumar interés o cobrar comisión), demostrando un diseño dinámico y limpio.

- **Reportes Globales:** Mediante el reporte_general(), el banco itera sobre todas las cuentas existentes para sumar y mostrar el patrimonio total gestionado.

```
#  OPERACIONES
    def transferir(self, numero_origen, numero_destino, monto):
        origen = self.buscar_cuenta(numero_origen)
        destino = self.buscar_cuenta(numero_destino)
        if origen is None or destino is None:
            print("Una de las cuentas no existe.")
            return False
        if origen == destino:
            print("No se puede transferir a la misma cuenta.")
            return False
        # Primero se intenta retirar; solo si funciona se deposita en el destino
        if origen.retirar(monto, "Transferencia enviada"):
            destino.depositar(monto, "Transferencia recibida")
            print(f"Transferencia exitosa de S/ {monto:.2f}: "
                  f"{origen.get_numero()} -> {destino.get_numero()}")
            return True
        return False

    def cierre_de_mes(self):
        print("\n=== CIERRE DE MES ===")
        for cuenta in self.__cuentas:
            saldo_antes = cuenta.get_saldo()
            cuenta.cierre_de_mes()          # POLIMORFISMO: cada cuenta hace lo suyo
            diferencia = cuenta.get_saldo() - saldo_antes
            print(f"  {cuenta.get_numero()} {cuenta.get_tipo():<17} "
                  f"cambio: S/ {diferencia:>8.2f}   nuevo saldo: S/ {cuenta.get_saldo():.2f}")
```

```
def reporte_general(self):
    total = 0
    for cuenta in self.__cuentas:
        total += cuenta.get_saldo()
    print(f"\n=== REPORTE GENERAL - {self.__nombre} ===")
    print(f"  Clientes registrados : {len(self.__clientes)}")
    print(f"  Cuentas abiertas     : {len(self.__cuentas)}")
    print(f"  Dinero total en banco: S/ {total:.2f}")
```

# CLASE CUENTA

La clase Cuenta modela una cuenta bancaria para gestionar el dinero de un titular, controlar saldos y registrar movimientos financieros de forma automática.

- __init__: Inicializa los datos de la cuenta, asignando un número único correlativo y un saldo inicial opcional.
- __str__: Define la representación en texto de la cuenta para mostrarla ordenadamente.
- depositar: Valida que el monto sea mayor a cero, aumenta el saldo y registra el movimiento.
- puede_retirar y retirar: Comprueban la disponibilidad de fondos antes de efectuar el retiro de dinero.
- estado_de_cuenta: Muestra un reporte completo con los datos del titular, el historial de transacciones y el saldo final.

```python
class Cuenta:

    contador = 1
    def __init__(self, titular, saldo_inicial=0):
        self.__numero = "001-" + str(Cuenta.contador).zfill(5)
        Cuenta.contador += 1
        self.__titular = titular
        self.__saldo = 0
        self.__movimientos = []
        if saldo_inicial > 0:
            self.depositar(saldo_inicial, "Apertura de cuenta")

    def __str__(self):
        return (f"{self.__numero}  {self.get_tipo():<17} "
                f" Saldo: S/ {self.__saldo:>10.2f}")

    def get_numero(self):
        return self.__numero

    def get_titular(self):
        return self.__titular

    def get_saldo(self):
        return self.__saldo

    def get_movimientos(self):
        return self.__movimientos

    def get_tipo(self):
        return "Cuenta"
```

```python
def __registrar(self, tipo, monto):
    self.__movimientos.append(Transaccion(tipo, monto, self.__saldo))

def depositar(self, monto, tipo="Deposito"):
    if monto <= 0:
        print("El monto a depositar debe ser mayor que 0.")
        return
    self.__saldo += monto
    self.__registrar(tipo, monto)
    return f"Deposito exitoso"

def puede_retirar(self, monto):
    if monto <= self.__saldo:
        return f"Retiro permitido"
    else:
        print(f"Saldo insuficiente")
        return False

def retirar(self, monto, tipo="Retiro"):
    if monto <= 0:
        print("El monto a retirar debe ser mayor que 0.")
        return
    if not self.puede_retirar(monto):
        print(f"Operacion rechazada: no se puede retirar S/ {monto:.2f} "
              f"de la cuenta {self.__numero}.")
        return
    self.__saldo -= monto
    self.__registrar(tipo, monto)
    return f"Retiro exitoso"
```

```python
def cierre_de_mes(self):
    pass

def estado_de_cuenta(self):
    print(f"\n===== ESTADO DE CUENTA {self.__numero} =====")
    print(f"Tipo    : {self.get_tipo()}")
    print(f"Titular : {self.__titular.get_nombre_completo()}")
    print("-" * 58)
    if len(self.__movimientos) == 0:
        print("  (sin movimientos)")
    for i in range(len(self.__movimientos)):
        print(f"  {i + 1:>2}. {self.__movimientos[i]}")
    print("-" * 58)
    print(f"  SALDO ACTUAL: S/ {self.__saldo:.2f}")
```

# SUBCLASE CUENTAAHORROS

El código de la subclase CuentaAhorros modela una cuenta orientada al ahorro pasivo que premia la retención de dinero con intereses e impone un límite de retiros mensuales.

- __init__: Invoca al constructor de la clase base Cuenta para registrar al titular y saldo, e inicializa de forma privada la tasa de interés y el contador de retiros del mes en cero.
- puede_retirar: Sobrescribe la regla de validación, autorizando la operación si el cliente no superó el tope mensual de 3 retiros y cuenta con saldo suficiente.
- retirar: Ejecuta el débito mediante la clase padre y, si la operación resulta exitosa, incrementa en una unidad el contador mensual de extracciones.
- cierre_de_mes: Calcula y abona automáticamente los intereses devengados sobre el saldo actual ("Intereses del mes") y reinicia a cero el contador de retiros.
- Getters (get_*): Permiten consultar de forma segura la tasa pactada (get_tasa_interes) y el número de retiros realizados en el periodo (get_retiros_del_mes).

```python
class CuentaAhorros(Cuenta):
    """Cuenta de ahorros: genera intereses mensuales y limita retiros a un máximo de 3 al mes."""

    LIMITE_RETIROS = 3  # Variable de clase que define el límite de retiros

    def __init__(self, titular, saldo_inicial=0, tasa_interes=0.01):
        # Reutiliza el constructor de la clase padre Cuenta
        Cuenta.__init__(self, titular, saldo_inicial)
        self.__tasa_interes = tasa_interes  # Tasa mensual (ej. 0.01 = 1%)
        self.__retiros_del_mes = 0
```

```python
    # Getters
    def get_tasa_interes(self):
        return self.__tasa_interes

    def get_retiros_del_mes(self):
        return self.__retiros_del_mes

    def get_tipo(self):
        """Sobrescribe el tipo de cuenta de la clase padre."""
        return "Cuenta de Ahorros"

    def puede_retirar(self, monto):

        if self.__retiros_del_mes >= CuentaAhorros.LIMITE_RETIROS:
            print(
                f"Límite de {CuentaAhorros.LIMITE_RETIROS} retiros por mes
                alcanzado."
            )
            return False
        return Cuenta.puede_retirar(self, monto)

    def retirar(self, monto, tipo="Retiro"):
        """Sobrescribe retirar: si la operación se concreta, incrementa
        el contador mensual."""
        exito = Cuenta.retirar(self, monto, tipo)
        if exito:
            self.__retiros_del_mes += 1
        return exito
```

```python
def cierre_de_mes(self):
    """Sobrescribe cierre_de_mes: abona los intereses acumulados y
    reinicia el contador de retiros."""
    interes = round(self.get_saldo() * self.__tasa_interes, 2)
    if interes > 0:
        self.depositar(interes, "Intereses del mes")
    self.__retiros_del_mes = 0
```

# SUBCLASE CUENTACORRIENTE

El código de la subclase CuentaCorriente modela una cuenta comercial transaccional que proporciona liquidez contingente mediante una línea de sobregiro autorizado, aplicando una tarifa fija mensual por mantenimiento.

- __init__: Reutiliza el constructor de la clase base Cuenta para inicializar el titular y el balance, y registra de forma protegida el margen de financiamiento o sobregiro asignado (por defecto S/ 500.00).
- puede_retirar: Sobrescribe la regla de validación contable, autorizando retiros aun cuando superen el saldo disponible, siempre que el monto solicitado no exceda el límite de sobregiro.
- retirar: Delega el procesamiento del débito a la clase base Cuenta, permitiendo balances negativos.
- cierre_de_mes: Aplica automáticamente la comisión fija de mantenimiento (S/ 10.00) bajo el concepto "Mantenimiento mensual" en el historial contable.
- Getters (get_*): Permite consultar el monto del cupo de sobregiro asignado a la cuenta mediante get_limite_sobregiro.

```python
class CuentaCorriente(Cuenta):
    """Cuenta corriente: admite sobregiro (saldo negativo) y cobra
       comisión mensual de mantenimiento."""

    COMISION_MANTENIMIENTO = 10  # Cobro fijo mensual (S/ 10)

    def __init__(self, titular, saldo_inicial=0, limite_sobregiro=500):
        # Reutiliza el constructor de la clase padre Cuenta
        Cuenta.__init__(self, titular, saldo_inicial)
        self.__limite_sobregiro = limite_sobregiro
```

```python
    # Getter
    def get_limite_sobregiro(self):
        return self.__limite_sobregiro

    def get_tipo(self):
        """Sobrescribe el tipo de cuenta de la clase padre."""
        return "Cuenta Corriente"

    def puede_retirar(self, monto):
        """Sobrescribe la regla: permite retirar incluso sin fondos,
           hasta el margen de sobregiro."""
        return monto <= (self.get_saldo() + self.__limite_sobregiro)
```

```python
    def cierre_de_mes(self):
        """Sobrescribe cierre_de_mes: debita la comisión mensual de
           mantenimiento."""
        self.retirar(
            CuentaCorriente.COMISION_MANTENIMIENTO, "Mantenimiento mensual"
        )
```

# 6. MENU CONSOLA

```python
def mostrar_titulo(banco):
    print("=" * 58)
    print(f"        SISTEMA BANCARIO - {banco.get_nombre().upper()}")
    print("=" * 58)

def leer_monto(mensaje):
    """Pide un monto. Si el usuario escribe algo que no es numero, devuelve -1."""
    texto = input(mensaje)
    if texto.replace(".", "", 1).isdigit():
        return float(texto)
    return -1

def leer_opcion(mensaje):
    texto = input(mensaje)
    if texto.isdigit():
        return int(texto)
    return -1

def cargar_datos_de_ejemplo(banco):
    """Crea clientes y cuentas listos para la demostracion."""
    banco.registrar_cliente("45678912", "Maria", "Quispe Rojas", "987654321")
    banco.registrar_cliente("71234567", "Carlos", "Mendoza Diaz", "912345678")
    banco.registrar_cliente("40112233", "Lucia", "Torres Vega", "956789123")
    banco.abrir_cuenta("45678912", 1, 1500)     # Maria - Ahorros
    banco.abrir_cuenta("45678912", 2, 800)      # Maria - Corriente
    banco.abrir_cuenta("71234567", 1, 3200)     # Carlos - Ahorros
    banco.abrir_cuenta("40112233", 2, 250)      # Lucia - Corriente
```

```python
41  def registrar_cliente(banco):
42      dni = input("DNI (8 digitos): ")
43      nombre = input("Nombres: ")
44      apellido = input("Apellidos: ")
45      telefono = input("Telefono: ")
46      banco.registrar_cliente(dni, nombre, apellido, telefono)
47
48  def abrir_cuenta(banco):
49      dni = input("DNI del cliente: ")
50      print("Tipo de cuenta:  1) Ahorros   2) Corriente")
51      tipo = leer_opcion("Elige: ")
52      monto = leer_monto("Deposito inicial (S/): ")
53      if monto < 0:
54          monto = 0
55      banco.abrir_cuenta(dni, tipo, monto)
56
57  def depositar(banco):
58      cuenta = banco.buscar_cuenta(input("Numero de cuenta (ej. 001-00001): "))
59      if cuenta is None:
60          print("La cuenta no existe.")
61          return
62      monto = leer_monto("Monto a depositar (S/): ")
63      if cuenta.depositar(monto):
64          print(f"Deposito exitoso. Nuevo saldo: S/ {cuenta.get_saldo():.2f}")
65
66  def retirar(banco):
67      cuenta = banco.buscar_cuenta(input("Numero de cuenta: "))
68      if cuenta is None:
69          print("La cuenta no existe.")
```

```
Problems  Output  Debug Console  Terminal  Ports

  1. Cargar datos de ejemplo
  2. Registrar cliente
  3. Abrir cuenta
  4. Depositar
  5. Retirar
  6. Transferir
  7. Ver estado de cuenta (movimientos)
  8. Ver clientes y sus cuentas
  9. Cierre de mes (intereses y comisiones)
 10. Reporte general del banco
  0. Salir
 Elige una opcion:
```

# 7. PRUEBAS

### Visualización de Clientes

El sistema muestra el nombre , DNI , telefono y el listado de productos de cada usuario

```
========== CLIENTES DEL BANCO NACIONAL DEL AHORRO ==========

Cliente: Maria Quispe Rojas  |  DNI: 45678912  |  Tel: 987654321
   001-00017  Cuenta de Ahorros  Saldo: S/    1500.00
   001-00018  Cuenta Corriente   Saldo: S/     800.00
   Saldo total: S/ 2300.00

Cliente: Carlos Mendoza Diaz  |  DNI: 71234567  |  Tel: 912345678
   001-00019  Cuenta de Ahorros  Saldo: S/    3200.00
   Saldo total: S/ 3200.00

Cliente: Lucia Torres Vega  |  DNI: 40112233  |  Tel: 956789123
   001-00020  Cuenta Corriente   Saldo: S/     250.00
   Saldo total: S/ 250.00
```

### Prueba de Transferencia

Mensaje de la consola confirmando el éxito de la transacción de cuenta a cuenta

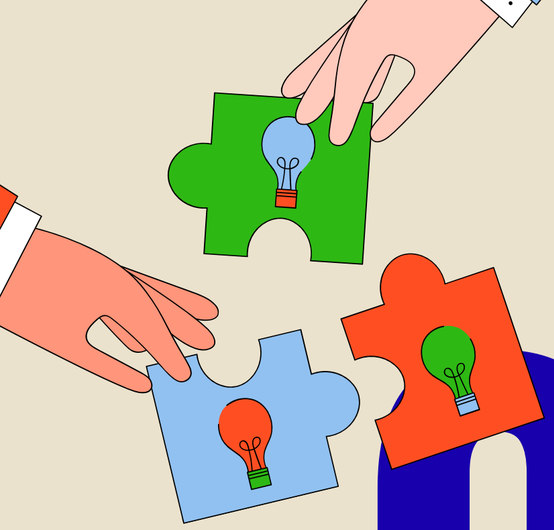
```
------------------ MENU ------------------
 1. Cargar datos de ejemplo
 2. Registrar cliente
 3. Abrir cuenta
 4. Depositar
 5. Retirar
 6. Transferir
 7. Ver estado de cuenta (movimientos)
 8. Ver clientes y sus cuentas
 9. Cierre de mes (intereses y comisiones)
10. Reporte general del banco
 0. Salir
Transferencia exitosa de S/ 100.00: 001-00017 -> 001-00020
```

# 7. PRUEBAS

### Cierre de Mes

La consola muestra cómo los saldos cambian dinámicamente según el tipo de cuenta

```
=== CIERRE DE MES ===
  001-00017 Cuenta de Ahorros cambio: S/    14.00   nuevo saldo: S/ 1414.00
  001-00018 Cuenta Corriente  cambio: S/   -10.00   nuevo saldo: S/ 790.00
  001-00019 Cuenta de Ahorros cambio: S/    32.00   nuevo saldo: S/ 3232.00
  001-00020 Cuenta Corriente  cambio: S/   -10.00   nuevo saldo: S/ 340.00
```



# GRACIAS

